

CSC 452/552 Operations Systems

Project 4 Buddy Allocator

Name: Sean Varie

Bronco ID: 114159195

Date: December 3, 2024

1. Project Overview

Project 4 was an implementation of the Buddy Allocator memory management algorithm. This algorithm involves splitting the memory space into powers of two. This minimizes external fragmentation and allows for some quite efficient handling of a number of the memory manager api calls. In addition to the implementation of this algorithm, this project involved the implementation of a number of tests for the memory manager.

2. Project Management Plan

a) Task 3: Implement Buddy

Task 3 was the implementation of the specified memory management api using the buddy algorithm. The algorithm was followed as described in class. One thing to take note of is that each linked list of available blocks has a dummy head at the beginning with its tag set to BLOCK_UNUSED. buddy_realloc was implemented such that if the currently inhabited block of the pointer given is big enough to accommodate the new size, then that pointer is returned and nothing is changed, otherwise, a new, big enough, block is allocated, the appropriate data copied and the old block freed. There are a few extra optimizations that could be added to buddy_realloc.

b) Task 4: Write Unit Tests

Task 4 was done in parallel with task 3. As I began writing different parts and functionality of the memory manager, I would write a test for that functionality. I ended up testing various situations of the memory manager in order to cover the different parts of the code.

3. Project Deliveries

a) How to compile and use my code?

There is no driver provided as part of this project, only the test suite. To compile and run the test suite, run the following command from the project directory:

```
$ make check
```

b) Any self-modification?

The only modifications to the provided code was the above mentioned implementation of the buddy algorithm and the addition of tests. I did make an addition to the provided testing method check_buddy_pool_empty. This addition ensures that a call to buddy_malloc for the given pool results in a null pointer being returned.

c) Summary of Results.

Running my test suite results in all tests passing:

```
@seanvarie → /workspaces/cs452-p1 (P4) $ ./test-lab
Random seed:1733453428
Running memory tests.
->Testing buddy init
tests/test-lab.c:344:test_buddy_init:PASS
->Test allocating and freeing 1 byte
tests/test-lab.c:345:test_buddy_malloc_one_byte:PASS
->Testing size that will consume entire memory pool
tests/test-lab.c:346:test_buddy_malloc_one_large:PASS
->Test attempting allocating too large a block
tests/test-lab.c:347:test_buddy_malloc_one_too_large:PASS
->Test allocating 1 byte, attempting allocating too large a block and freeing the 1 byte
tests/test-lab.c:348:test_buddy_malloc_one_byte_and_one_too_large:PASS
->Test allocating and freeing a few blocks of memory
tests/test-lab.c:349:test_multiple_buddy_malloc:PASS
->Test data is retained from allocating and reallocating
tests/test-lab.c:350:test_buddy_realloc_data:PASS
->Test allocating 2 bytes and reallocating as 1
tests/test-lab.c:351:test_buddy_realloc_2_to_1_bytes:PASS
->Test allocating 1 byte and reallocating as 2
tests/test-lab.c:352:test_buddy_realloc_1_to_2_bytes:PASS
->Test allocating, reallocating and freeing a few blocks of memory
tests/test-lab.c:353:test_multiple_buddy_malloc_and_realloc:PASS

-----
10 Tests 0 Failures 0 Ignored
OK
@seanvarie → /workspaces/cs452-p1 (P4) $
```

4. Self-Reflection of Project

This project was fairly straightforward, every detail of the algorithm was basically handed to us. While I didn't directly translate every detail from the paper describing the algorithm, it, along with in class explanations, gave me enough information about how the algorithm was meant to work that I was easily able to implement it. When testing, I ran into a few bugs due to some oversights or silly mistakes that I made when writing the code, but my strategy of interweaving testing with incremental development made finding and fixing these bugs quick and easy.